

Docker Documentation

Node.js + Express Application

الطالبة :

نسمة عبدالقادر علوي بن شهاب

Introduction

في هذا العمل تم استخدام Docker لبناء وتشغيل تطبيق Node.js مع Express داخل بيئة معزولة باستخدام Containers. تم تنفيذ جميع الخطوات عبر Command Line Interface (CLI) دون الاعتماد على واجهة رسومية، لمحاكاة بيئة العمل على السيرفرات. شمل العمل إنشاء Dockerfile لبناء صورة مخصصة للتطبيق، ثم تشغيلها داخل Container مع استخدام Volumes لربط ملفات المشروع وتمكين خاصية Hot Reload أثناء التطوير. كما تم تجربة تحميل صور جاهزة من Docker Hub باستخدام أمر `docker pull` وأخيراً، تم استخدام أداة Docker Compose لتسهيل إدارة وبناء وتشغيل الكونتainers، حيث تم استبدال الأوامر الطويلة بأمر واحد يحقق نفس الوظيفة بكفاءة أعلى.

Step 1: install docker

طبعاً سيتم إنشاء الكونتير وكل الخطوات عن طريق أوامر command line لان في المستقبل اذا اشتغلنا على الserver ما يكون عندنا رفاهية UI

Step 2: Create node JS application (node JS اختياري انا أنشأت)

Step 3: Install Dependencies

تم تثبيت المكتبات المطلوبة

لإنشاء السيرفر Express

لتسهيل التطوير وإعادة التشغيل التلقائي Nodemon

Step 4: Write Simple Application

تم إنشاء تطبيق Express بسيط داخل الملف `src/index.js` يقوم بتشغيل سيرفر على المنفذ 3000.

Step 5: Add Scripts to package.json

تم إضافة سكريبتين لتشغيل التطبيق:

```
"scripts": {  
  "start": "node src/index.js",  
  "start-dev": "nodemon src/index.js"  
}
```

بعد ما انشأنا المشروع الان سنبدأ بخطوات استخدام الدوكر في هذا المشروع

Step 6: Install Docker Extension

تم تثبيت Docker Extension في VS Code لتسهيل كتابة أوامر Docker وإدارة الملفات.

Step 7: Create Dockerfile

Step 8: Write Dockerfile Instructions

```
FROM node  
WORKDIR /app  
COPY package.json .  
RUN npm install  
COPY . .  
EXPOSE 3000
```

CMD ["npm", "run", "start-dev"]

شرح الأوامر:

FROM node: Base Image تحديد الـ

WORKDIR /app: إنشاء مجلد app داخل الكونت이너 للتنظيم

COPY package.json : نسخ ملف package.json إلى الكونت이너

RUN npm install: تثبيت node_modules داخل الكونت이너

COPY . . : نسخ جميع ملفات المشروع

EXPOSE 3000: توضيح أن التطبيق يعمل على المنفذ 3000

CMD: تشغيل التطبيق عند تشغيل الكونت이너

Step 9: Build Docker Image

لبناء الصورة نستخدم الأمر:

```
docker build -t express-node-app . // نستخدمها عشان نعطي اسم للصورة اللي بيتم إنشائها
```

وللتأكد من إنشاء الصورة:

```
docker image ls // يعرض لنا كل الصور
```

Step 10: Run Docker Container

لتشغيل الصورة وإنشاء كونت이너:

```
docker run --name express-node-app-container \
```

```
-v <full-path>/src:/app/src:ro \
```

```
-p 3000:3000 express-node-app
```

شرح الأمر:

--name: إعطاء اسم للكونت이너

-v: ربط ملفات المشروع (Hot Reload)

-p: ربط منفذ الجهاز المحلي بمنفذ الكونت이너

لعرض الحاويات:

```
docker ps
```

Step 11: Create .dockerignore File

تم إنشاء ملف: .dockerignore

لاستثناء الملفات غير الضرورية مثل: node_modules

Step 12: Pull Ready Image from Docker Hub

تم تجربة تحميل صورة جاهزة باستخدام الأمر:

```
docker pull <image-name>
```

Step 13: Using Docker Compose utility

هذا يسهل لنا إدارة الكونت이너 ويعطينا ميزات

Create docker-compose.yml

```
version: '5.0.2'

services:
  node-app:
    container_name: express-node-app-container
    build: .
    volumes:
      - ./src:/app/src:ro
    ports:
      - "3000:3000"
```

Run Docker Compose

لتشغيل التطبيق باستخدام Docker Compose

`docker-compose up`

هذا الأمر يقوم ببناء الصورة وإنشاء وتشغيل الكونتiner بدل كتابة الأوامر الطويلة يدويًا

GitHub Documentation

Creating a Shared Repository and
Working with Branches

Introduction

في هذا العمل تم استخدام GitHub لإنشاء مستودع (Repository) جماعي يتيح التعاون بين أعضاء الفريق. تم تطبيق مفهوم Branches بحيث يعمل كل عضو على فرع خاص به دون التأثير على الفرع الرئيسي (Main) كما تم رفع ملفات PDF داخل الفرع الخاص بكل عضو.

Step 1: Create a New Repository on GitHub

1. تسجيل الدخول إلى GitHub
2. الضغط على زر New Repository
3. إدخال اسم المستودع
4. اختيار Public Repository
5. الضغط على Create Repository

Step 2: Invite Team Members

1. الدخول إلى صفحة المستودع
2. الضغط على Settings
3. اختيار Collaborators
4. إضافة أعضاء الفريق باستخدام اسم المستخدم أو البريد الإلكتروني
5. قبول الدعوة من قبل الأعضاء

Step 3: Clone the Repository

على الجهاز المحلي، يتم نسخ المستودع باستخدام الأمر:

```
git clone <repository-url>
```

ثم الدخول إلى مجلد المشروع:

```
cd repository-name
```

Step 4: Create a Personal Branch

يقوم كل عضو بإنشاء فرع خاص به للعمل عليه:

```
git checkout -b nesma-branch
```

Step 5: Add PDF Files to the Branch

1. يتم إضافة ثلاثة ملفات PDF داخل مجلد المشروع
2. ثم تنفيذ الأوامر التالية:

```
git add .
```

```
git commit -m "Add PDF files to personal branch"
```

Step 6: Push the Branch to GitHub

لرفع الفرع إلى GitHub

```
git push origin nesma-branch
```

Conclusion

يساعد استخدام GitHub Repository المشترك و Branches على تنظيم العمل الجماعي، ومنع تعارض التعديلات بين أعضاء الفريق. كما يتيح لكل عضو العمل بحرية على ملفاته الخاصة مع إمكانية مراجعتها أو دمجها لاحقًا عند الحاجة.